

CODING BEST PRACTICES IN-LECTURE GITHUB EXERCISE

Introduction

This document contains the instructions for completing the hands-on exercises during the git/github portion of the Coding Best Practices/Data Visualization lecture. The objective of this exercise is to give you hands-on experience with key git concepts, including branching, committing, pushing, and pulling — all using GitHub and the command line. The primary focus is on practicing how git works in collaborative workflows.

Most steps in this exercise will be done individually, but everyone at your table will be working on the same repository, and so you'll need to collaborate in order to complete the overall exercise! The goal is for you to fully understand what's happening at every step, not just complete the activity.

- If you have less coding or git experience, ask questions and ensure you're not lost at any stage. Don't let the group move on unless you're clear on what just happened.
- If you have more experience, help guide your group by explaining what each git command or action does. Teaching will help reinforce your own understanding.

If your group gets stuck, raise your hand — I'll be happy to help as soon as I can.

Note: You should have already cloned the repo and authenticated in git, but if you did not do that, instructions are at the bottom of this document in the appendix. We will not be stopping for questions about this specifically.

Note 2: See some common terminal commands [here](#)

Part 1: Collaborating in Git – A Warm Up

Step 1: Verifying your repository and inspecting the contents

1. Check the folder contents using `ls` and inspect the Git status using `git status`.
 - a. The first command shows files already in the repo while the second tells you if any files have been modified or are ready to commit. You should see the files and folders included in the starter repo, including `README.md`, and some code and data files. `git status` should confirm there are no uncommitted changes yet.

Step 2: Make individual changes and commit

1. Open the `README.md` file in a text editor.
 - a. On a Mac, type `nano README.md`
 - b. On a PC, type `notepad README.md`
 - The first line of the file should read:

```
Git/Github README: YourName
```

3. Replace `YourName` with your actual name. Then save the file as `README_YOURNAME.md`. Now you'll

4. In your terminal, stage, commit, and push the change:

```
git add README_YOURNAME.md
```

```
git commit -m "Add first name to personal README"
```

----- BACK TO LECTURE -----

Step 3: Collaborating through Github

Now all group members should push the latest changes to GitHub:

1. Run the following command to push:

```
git push origin main
```

2. Now everyone can pull the rest of the group's readmes:

```
git pull origin main
```

----- BACK TO LECTURE -----

Step 4: Try out a branch workflow

1. **All group members** should now start their own branch:

```
git checkout -b your-name-branch
```

2. Then open another group member's readme in your text editor, and add the following line: "Peer reviewed by: YOUR NAME"
 - a. **Note:** To keep things simple, make sure everyone at the table is working on a different readme
 - i. If you want a challenge, try having more than one person work on the same readme at a time!
3. Add and commit the readme – you should be a pro at this now!
4. Now we want to push our dev branch:

```
git push origin your-name-branch
```

5. Then go to the github repo online and submit a pull request
 - a. If everyone worked on a different readme, then you should be able to merge the branch and close the pull request without any conflicts!
 - b. If more than one person worked on the same readme, then you'll need to resolve the conflict before closing the requests
6. Once your pull request is merged (or denied!), delete your branch
7. Then you can checkout back to main in your local repo:

```
git checkout main
```

ALMOST AT THE REAL EXERCISE WOO

----- BACK TO LECTURE -----

Appendix: Pre-Training Setup (to be done before the session)

1. **Install git:** If not already installed, install git on your computer. Follow the instructions found at <https://git-scm.com/downloads> for the operating system you use on the laptop you will bring to the training.
2. **Create a GitHub account:** If you don't already have one, sign up for a free account at <https://github.com>.
3. **Email your GitHub username** to Jack at jcavanagh@povertyactionlab.org once your account is set up.
4. **Join the GitHub Organization:** You will be invited to a GitHub organization created for the exercise. Once you accept the invitation, you'll be added to a private repository. You will need to access this during the training.
5. **Authenticate and clone the repository**
 1. Open the terminal on your computer.
 - a. Mac Users:
 - Navigate to the folder where you want to store the remote repository.
 - Right-click the folder and select "New Terminal at Folder" (or open Terminal and use `cd` to navigate to your desired location).
 - b. Windows Users:
 - Navigate to the folder where you want to store the remote repository.
 - Right click and select "Open in Terminal"
 2. Then, go to the repository page on GitHub. Click the green <Code> button and copy the HTTPS link (it will look like `https://github.com/...`).
 3. In your terminal, type the following (replacing the URL with the one you copied):

```
git clone https://github.com/your-org/your-repo.git
```
 4. If prompted for your GitHub username, enter it.
 5. If prompted for a password **do not use your GitHub password**.
 - a. Instead, generate a [personal access token \(PAT\)](#) from your GitHub account.

- b. Select scopes that include repo access, and then copy the token and paste it in as your password.